Project Deliverable 3: Project Checkpoint

CSE 4059: Applied Parallel Programming using GPUs

Group Members:

- Member 1: Charlie Li

- Member 2: Jack Yang

1. Progress Update

- Current Progress:

We have implemented the sequential algorithm and the force matrix kernel for force calculations successfully, and verified the consistency of the computation results. In addition, we implemented a straightforward kernel that maps threads to each atom and calculates the net force on it. We measured the empirical time taken by the sequential algorithm and the GPU kernels and calculated the speedup.

For the second method we mentioned in the project plan, we also implemented two CUDA kernels and one intermediate function to successfully compute the result. The details will be explained in the later section. Now it uses some random parameters which might not be the most efficient configuration.

- Comparison to Plan:

We are on track with the plan. We have implemented the force matrix algorithm and the cutoff algorithm and ensured its correctness by comparing its results to the sequential algorithm to the fifth decimal place. We implemented an additional kernel with a simpler design, and compared its speedup to the force matrix kernel. We have started to measure the run times of the algorithms but in a naive way. More sophisticated and fine-grained experiments need to be designed and benchmarked in the next two weeks.

2. Implementation Details

- Implementation:

To allow for memory coalescing for the GPU kernel, we structured the global positions, forces and velocities arrays in the layered format (with shape 3 * n_particles). We implemented all algorithms based on this data structure.

Method 1:

The sequential algorithm is implemented with a doubly nested loop, which calculates forces for each unique pair once and accumulates them to the global forces array. There are in total $\frac{n(n-1)}{2}$ force calculations and $n(n-1)$ global memory writes performed in this algorithm, and its time complexity is $O(n^2)$, where n is the number of particles.

The segmented traversal algorithm is a straightforward GPU implementation of force computation. In this algorithm, each thread is mapped to one atom, and it loops through all the other atoms and adds the forces into a local register variable. Finally, it stores the net force on that atom into the global forces array. Since every atom needs access to the positions of every other atom, there are lots of memory reuse opportunities. We optimized this algorithm by having threads in one block cooperate to load segments of the positions array into shared memory and calculate forces using the loaded positions. The span of each thread is n, and the total computation work performed by this kernel is n^2. The number of thread blocks is $\frac{n}{tileDim}$, the total number of global memory reads is $\frac{n^2}{tileDim}$, and the number of global memory writes is n.

Although the segmented traversal kernel is clean and achieves impressive speedup, it wastes half of the computation work, since according to Newton's Third Law, the magnitude of the forces are the same between a pair of atoms. The force matrix kernel attempts to reduce unnecessary computations by paying the cost of more global memory writes and contentions. We envision a $n \times n$ force matrix, with each entry recording the force from the row atom (source) to the column atom (receiver). We only need to calculate the upper triangular part of this matrix, and the lower triangular entries can be filled by taking the inverse of the entry at its transposed position. In a naive implementation, we could launch a kernel to fill this matrix, then perform parallel reduction to sum the forces. However, we will need at least a memory size of $n^2\ entries \times 3\ dimensions/entry \times 8\ bytes/dimension = 24n^2\ bytes$. A global memory of $8GB$ can hold at most 18918 atoms, which is sufficient for the simulation of an isolated linear polymer but does not scale well for larger systems (for example if solution molecules are modeled). We also pay the price of n^2 global memory writes.

To save on space utilization, we broke the matrix into tiles and mapped blocks to tiles. Each block maps to a tile of $t \times t$ atoms in the matrix but only has t threads. We accumulated the forces for each row atom and each column atom in registers or shared memory before adding them back to the global forces array. For each block, we loaded the positions of the receiver atoms into registers and the positions of the source atoms into shared memory. Each thread then loops through the sources and adds the forces into a register accumulator variable, as well as the inverse of that force into shared memory. To minimize memory contention, we juxtaposed the source atom indices processed by each thread in each iteration. In the ith iteration, thread k will process the force of atom $(i + k)\%t$ on atom k. When $t \leq 32$, there will be no shared memory contention, since threads in each warp are operating on different positions in shared memory. There are $\frac{(1+\frac{n}{t})\frac{n}{t}}{2}$ blocks, with $\frac{n}{t}$ diagonal blocks and $\frac{(\frac{n}{t})^2-\frac{n}{t}}{2}$ non-diagonal ones. The span of each thread is t, and the total computation work is $t^2 \times \frac{(1+\frac{n}{t})\frac{n}{t}}{2} = \frac{(n+t)n}{2}$. The total number of global memory reads is $2t\frac{(\frac{n}{t})^2-\frac{n}{t}}{2} + t\frac{n}{t} = \frac{n^2}{t} - n + n = \frac{n^2}{t}$, and the total number of global memory writes is the same as the number of reads. However, we pay the price of memory contention by not storing the whole matrix. All $\frac{n^2}{t}$ global memory writes are performed with atomic operations, with at most $\frac{n}{t}$ writes made to the same positions.

Method 2:

The overall idea of the cutoff-distance method is to divide the simulation box into many small cells, where each cell may contain multiple atoms or may be empty. Then for one atom inside a home cell, we do not compare it with all other atoms in the system. Instead, we only check atoms inside its own cell and the 26 surrounding cells, so in total 27 cells. After that, even for atoms inside those candidate cells, we still only compute the interaction if the actual pair distance is within the cutoff. In our current implementation, the cutoff is 10.0. Since the Lennard-Jones interaction becomes very small when the distance is large, this assumption lets us ignore many pairwise computations without changing the result too much. Because of this, the force evaluation is much cheaper than the original all-pairs method.

The whole method can be divided into three phases: **Label**, **Sort**, and **Compute**.

The first phase is the **Label** phase. In this phase, we assign one cell label to each atom based on its current position. Suppose the box is divided into **`dim`** cells in each direction, and each cell has width **`cell_size`**. Then for one atom, we compute **`cell_x`**, **`cell_y`**, and **`cell_z`** by dividing its coordinate by **`cell_size`** and taking the integer part. Since positions are already inside the box, the cell index in each direction is nonnegative. Then the 3D cell index is converted into one 1D label by **`cell_x + cell_y*dim + cell_z*dim*dim`**. We store these labels in a temporary array of size **`n`**, the number of atoms. This phase is very suitable for parallelism because each atom is processed independently, and there is no communication or race condition between threads.

The second phase is the **Sort** phase. In this phase, we sort atoms according to their cell labels, so atoms that belong to the same cell become consecutive in memory. After sorting, we reorder the position array in the same way, so the new position array is grouped by cell. Then we count how many atoms belong to each label by a histogram operation, and after that we do a prefix sum on this count array. This gives an array called **`cell_start`**. The meaning of **`cell_start[c]`** is the starting index of cell **`c`** in the sorted position array, and **`cell_start[c+1]`** is the ending index. Therefore, all atoms inside cell **`c`** are exactly in the range from **`cell_start[c]`** to **`cell_start[c+1]-1`**. This makes neighbor-cell lookup very efficient, because once a neighboring cell id is known, we can directly jump to the corresponding range in the sorted array. One issue in this phase is that after sorting, the atom order is no longer the original chain order from the initial position array. Because of this, we also store another array called **`org_idx`**, which records the original index of each atom before sorting. This is useful in our case because the force model depends not only on distance but also on the separation along the chain, for example distinguishing bonded, near-bonded, and nonbonded interactions. If the original order were not needed, then this method would be even more favorable, because after one timestep the positions are already partially grouped in space, which may reduce the sorting work in the next iteration. In our implementation, this whole phase is done by **CuPy** functions such as **`argsort`**, **`bincount`**, and **`cumsum`**. These operations are already highly optimized, so it is not necessary to manually write extra GPU kernels for them.

The last phase is the **Compute** phase. In this phase, each atom is mapped to exactly one thread, and that thread computes the total net force acting on this atom. This design avoids race conditions, because each thread only writes to one force entry. The kernel first finds the home cell of the current atom, then loops over the 27 neighboring cells including the home cell. For each neighboring cell, it gets the corresponding atom range from **`cell_start`**, then iterates through all atoms in that range. For each

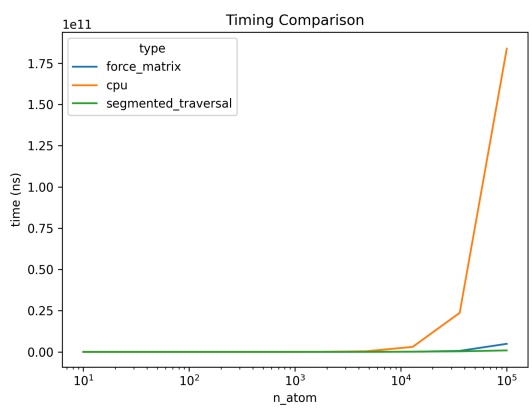
candidate atom pair, it computes the minimum-image displacement, then the pair distance, and if the distance is smaller than the cutoff, it evaluates the force and adds it to the local force accumulator. At the end, the thread writes the final **`x`**, **`y`**, and **`z`** force components back to global memory according to the index stored in **`org_idx`**.

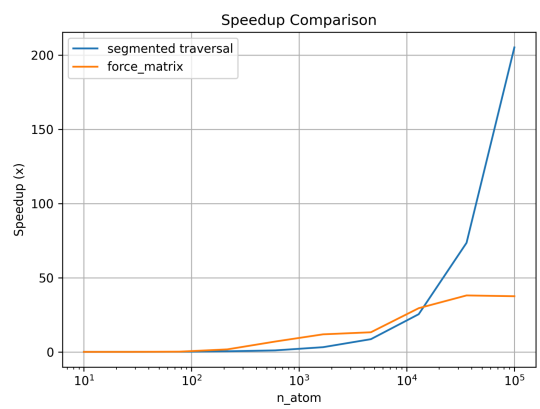
The sequential version follows almost the same logic as the parallel version. It still has the same three phases: labeling atoms, sorting by cell, building **`cell_start`**, and then searching only over neighboring cells. The main difference is that all these operations are done in sequential loops instead of parallel GPU kernels or CuPy primitives. Therefore, the sequential version is mainly used as a reference implementation to verify correctness and compare runtime.

- Performance Measurements:

Method 1:

We measured the empirical run time of the different algorithms with n ranging from 10^1 to 10^5 by taking the average between 5 runs. Using the sequential algorithm's run time as a baseline, we computed the speedup achieved by our GPU kernels and plotted them against n. The plots are shown below.

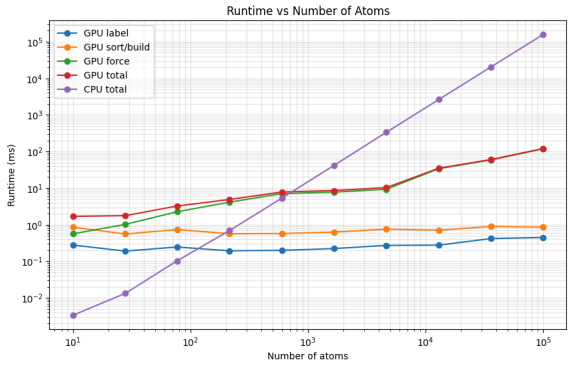


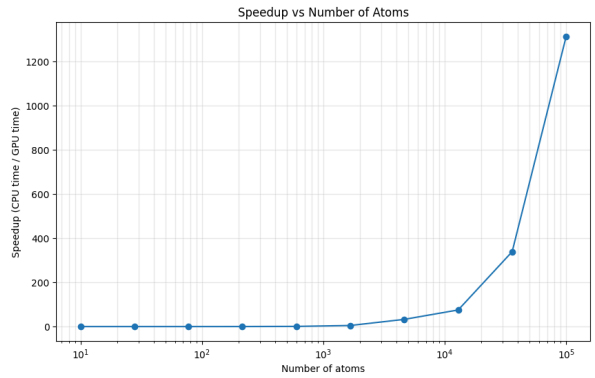


Method 2:

We used the similar setting compared to method 1 except that there is no naive kernel that does simple parallelism of the work. The results below have shown that the cutoff method performs well when the number of atoms in a chain is large. Before the number of atoms reaches about 1000, the CPU method has a shorter runtime than the GPU method. However, when the number of atoms is larger than the scale of 10^3, the GPU algorithm outperforms the CPU algorithm, which is desirable for the real-world

application.





Provide a link to a GitHub repository containing your project code.

https://github.com/P-mandevillei/CUDA_Molecular_Dynamics

3. Revisions to Project Plan

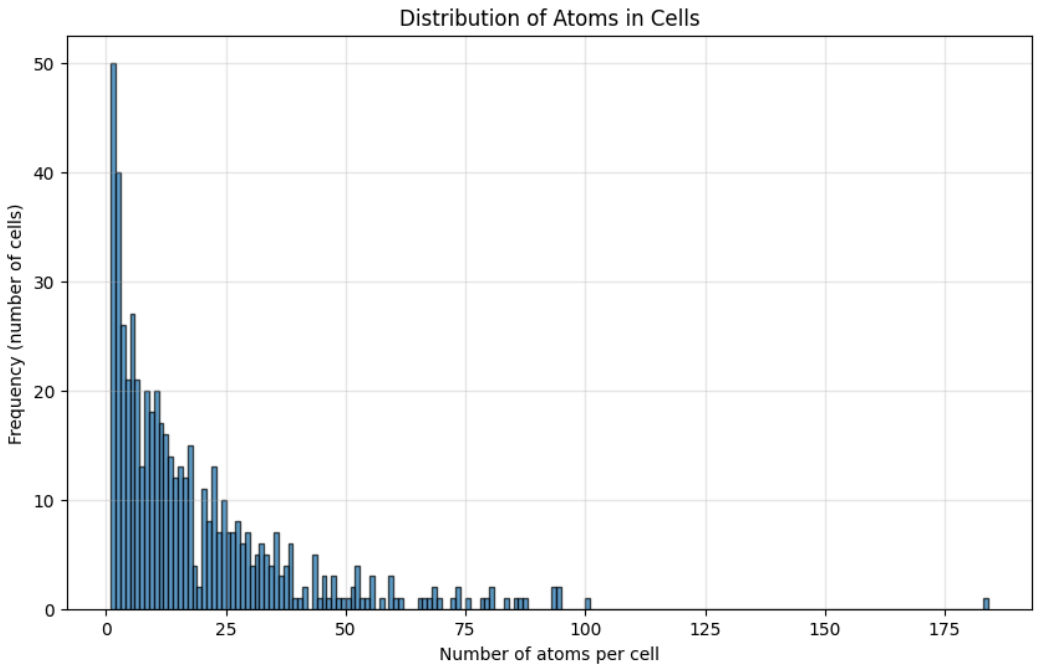
- Revisions:

We implemented the segmented traversal kernel in addition to the two kernels we planned to implement. It would also be interesting to implement the force matrix kernel that actually fills out the matrix before performing parallel reduction, since it could be faster for smaller systems and avoids atomic adds.

In the current version of the cutoff method, we didn't implement the shared memory as we proposed in the project plan. Rather we just do the simple global read for each pair of atoms. During the process of designing the algorithm, we discovered that this highly imbalanced atom per cell is very similar to the sparse matrix case, where each entry may only contain a little useful information. It will be interesting to utilize some of the formats we discussed in last week's lecture to store our position information if time allows.

- Justification:

Although the force matrix kernel reduces total computation by half, it uses heavy atomic additions that scale quadratically with the number of atoms, which limits its run time on larger systems. We hypothesized that the segmented traversal kernel would provide greater speedup on larger systems, since it does not require atomic operations. Furthermore, given the limitations of the current force matrix kernel, it could be worth it to pay the price of global memory usage in order to accelerate the computations by filling out the entire matrix. Although this strategy will not accommodate systems larger than 10,000 atoms, it might work well with a moderate system size.

The main reason for not using shared memory in the cutoff method is that the occupancy of cells is very uneven. Most cells are empty, while a small number of cells contain many atoms. Because of this, it is difficult to design a balanced block-level scheme where one block handles one cell and each thread

handles one atom inside that cell. In that kind of design, many blocks would be almost empty, while some blocks would still have too much work. So for now, we use the simpler design where one thread handles one atom and directly reads candidate neighbor atoms from global memory. This is easier to implement and avoids complicated load balancing issues.